

Automated Muskmelon Counting in Real Farm Environment using YOLOv11n

Fourré Louis
CESI École d'ingénieur
France
fourre.louis@gmail.com

Boon-Yaik Ooi
Department of Computing
Universiti Teknologi PETRONAS
Malaysia
boonyaik@utp.edu.my

K. Sugathan Savita
Department of Computing
Universiti Teknologi PETRONAS
Malaysia
savitasugathan@utp.edu.my

Abstract— Fruit detection and counting in real farm environments remain challenging due to occlusion, variable lighting, and cluttered backgrounds. This study presents a robust computer vision pipeline for accurately detecting, tracking, and counting melons using the You Only Look Once version 11 nano object detection model. To evaluate and enhance performance, multiple dataset versions with different types of annotation (large annotation, front-row strict annotation, and front-row/rear-row annotation distinction) were utilized alongside various image processing pipelines (Roboflow workflows, detection, tracking, and line-counting tracking). Among these, the 4th method, with a line-counting strategy, significantly improved counting accuracy. Our best result was achieved with the front row strict annotation and line counting tracking combination which yielded a counting error rate of just 2.74%. The system was tested on footage captured by an automated guide vehicle in a greenhouse, demonstrating its practical applicability in real-world, non-laboratory agricultural environments. Overall, this work contributes an efficient and accurate melon counting pipeline, laying the foundation for future work such as fruit maturity prediction and disease detection.

Keywords— Computer Vision, YOLO, Crops recognition, Object Tracking

I. INTRODUCTION

Computer recognition in the agricultural field could become an important system to enhance productivity and reduce waste of farms. With computer recognition, farmers could monitor their plants and take decisions based on accurate footage and analytics of the past days or month. Monitoring data can help farmers in making decisions such as making decisions on the optimal moment to have the best harvest, detect and contain diseases, estimate the yield of the harvest and make optimal operational decisions such as advance logistics planning.

However, before taking any decisions, there must be a way to detect and at least to count the fruits in the fields. Unfortunately, farms are often not the best environment to have computer recognition. Computer vision struggles when the background is not uniform and in contrast of the subjects. Background of farms are often full of the same crops at a near distance on a different row and often be detected as well as those at the row in front of the camera. The recognition of crops can also be obstructed by themselves, when there are leaves in front of the flower/fruits.

This work proposes a model and a counting solution based on video footage of a melon greenhouse from automated guide vehicle (AGV). The proposed system is perfect to cases when it comes to recognition and detection of other fruits and vegetables in the farm that are suspended similar to the muskmelon. Meaning that melons will have a different height, like grape-based plants (e.g.: Tomato), improving the chance

to be switchable to other types of crops. The background is also very saturated and confusing due to plants going from the floor to the ceiling and with multiples rows in sight.

In this study, the proposed solution was evaluated against standard YOLOv11 methods, yielding reductions in double counting, enhanced isolation of back-row fruits, and an increase in counting accuracy to 97.26%. The processing time measured 153.09 s for an 85 s video on a local machine, indicating the partial applicability of the method.

II. LITERATURE REVIEW

Computer vision has increasingly found practical applications in agriculture, ranging from plant identification to maturity estimation or disease recognition. Among the many architectures used in this field, YOLO (You Only Look Once).

Zan et al. [1] proposed an improved YOLOv8 model (GSSS-YOLO) to classify tobacco leaves into maturity stages (under-ripe, ripe, and over-ripe). With a dataset of around 3,700 annotated images, their model achieved a precision of 93.3% and a recall of 92.4%, demonstrating YOLO's relevance in the precise detection of crops in real-world conditions.

Plant identification is another common use of computer vision in agriculture. Zhou et al. [2] developed a system based on ResNet-50 and NTS-Net to recognize Tibetan medicinal plants across 70 species, achieving a 93.3% success rate on over 13,000 images, emphasizing the effectiveness of attention-based feature localization in field conditions.

In addition, Faisal et al. [3] applied CNN models such as ResNet, VGG-19, and Inception-V3 to estimate the type, maturity, and weight of date fruits. Their system combined deep learning with Support Vector Machines (SVMs), achieving over 99% accuracy in classification tasks and 84.27% in weight estimation, highlighting the potential of hybrid models in multi-attribute recognition.

Disease recognition is another area where computer vision has shown promise. Liu et al. [4] presented a three-stage framework for plant disease identification involving region reweighting and feature integration. Trained on over 150,000 images spanning 271 plant diseases, the model reached a 90.01% success rate, although its complexity limited real-time deployment.

Barburiceanu et al. [5] explored an alternative approach by using pretrained CNNs such as AlexNet, VGG16, and ResNet variants for leaf texture feature extraction. Classification was performed using SVMs, enabling high accuracy with smaller datasets and reduced training requirements.

Computer vision has also been integrated into robotic systems. Hsia et al. [6] implemented a deep learning-based leaf detection system using ResNet-50 to automate a leaf-

blowing robot. Despite environmental noise such as grass and debris, the system achieved a 92% recognition rate, demonstrating the applicability of vision-based automation in outdoor agricultural tasks.

Beyond object classification, some studies addressed the complexity of agricultural environments. Akbar et al. [7] reviewed the use of deep learning in smart greenhouses, pointing out the challenges caused by complex and cluttered backgrounds and the lack of standardized datasets. They also underlined that embedded systems often struggle with real-time inference due to hardware limitations.

While many prior works focused on recognition tasks such as maturity classification or disease detection, few have explored object tracking and counting in real-world environments. This project distinguishes itself by implementing a YOLOv11n model combined with tracking and line-counting strategy. These methods aim to address the problems of occlusion, detection duplication, and background clutter, common challenges noted in the literature but seldom directly addressed in counting applications.

III. METHODOLOGY

The proposed work is based on the make of multiples versions and the usage of two different tools for the processing, Roboflow[8] and Ultralytics[9]. Roboflow will be to test if a cloud solution to compute the image could be viable, and Ultralytics will be to run the models on a local machine.

In addition to that, there are several annotation strategies tested to investigate their impact on detection accuracy. Each test is conducted using the same video, which has a duration of 1 minute and 25 seconds and contains 73 melons in the front row to be detected. The test data is video footage that is not used to training. The video consists of 73 muskmelons and the duration of the video is 85 seconds.

Due to the scarcity of muskmelons datasets online and due to limits in number of greenhouses in the area, only 2 videos were relevant enough to be used. On the 2 videos, one is used to make the dataset while one is used for testing and evaluation purposes.

All local experiments are performed on a laptop equipped with an NVIDIA RTX 4060m 8GB GPU and an Intel i5-12450HX processor.

A. Datasets Creation

To train a reliable melon detection model, custom datasets were created using Roboflow. The source material consisted of a single video from which only 187 relevant images were extracted.

During the project, multiples versions were created:

- V1D: Every visible melon in each image was annotated, with at least 75 percent of the melon visible and regardless of its position.
- V2D: Only melons in the front row that were clearly visible and mostly unobstructed by leaves were annotated. This version aimed to reduce false positives by focusing only on easily detectable fruits.
- V3D: Melons were annotated in two separate categories: front-melon and rear-melon. This version also included melons that were partially obscured, at least 50% of the fruit needed to be visible. This approach was designed to

test whether classifying by position would improve detection accuracy.

For each version, the images were split into 3 categories, 70% for training, 20% for validation and 10% for testing. Of the 187 images, there are 131 images for training, 37 for validation and 19 for testing. To increase the detection accuracy and detection capability, different effects were applied to training images:

- Brightness: Between -10% and +10%
- Exposure: Between -10% and +10%
- Outputs per training example: 3

The brightness and exposure effects serve to increase the chance of detection when the lights level varies.

This means that there are now 392 images for training, for a total of 448 images in the dataset. With this custom dataset, it is possible to use You Only Look Once version 11 nano (YOLOv11n) to create a custom model.

B. Running the model

The first running version (V1R) of the trained model remained hosted on Roboflow. For images, the model can be used directly within Roboflow web interface via its workflow. However, when applying the model to video data, a Python script is required to process the video frame by frame and then send each frame to the workflow.

To process the video, each frame is extracted from the video and sent via the Roboflow API to the workflow, returning the frame with annotations. The annotated frame is saved on the local computer to reassemble the video once every frame is treated

This version presents some inconveniences, need to be online, slow to process and take more space on disk than the other solutions. Plus, when moving from Roboflow workflow to Ultralytics library, the model couldn't be exported from Roboflow due to the free plan. Only the dataset can be downloaded, and the training must be redone on the local computer.

The V2R version is made with the 'predict' method from Ultralytics library. The video treated is split thanks to the OpenCV library. The 'predict' method is run on each frame to detect all the melons on it and add them to a counter. This version does not do persistence between the frames. It is very common for a single melon to be counted multiple times.

The V3R version is made with the 'track' method. The video is not split with OpenCV because the track version natively supports videos as an input. The 'track' method will track melons between frames, limiting the amount of duplication while counting the melons. This version of running the model will keep track of melons until they leave the view of the camera. But, if a melon is suddenly obstructed by a foreign object for too long the AI will lose tracking, and if it reappears later in the video, the ID of the melon will be different due to being considered as a new melon while it isn't. This issue can cause multiples case of duplication when counting.



FIG 1. EXAMPLE OF MELON DETECTION

To address this issue, the V4R uses the Solutions library from Ultralytics with the counter method. This method will count the objects of melons when they cross a line. This will ensure that the melons are counted only once, as long as they are visible and detected when they cross the line.

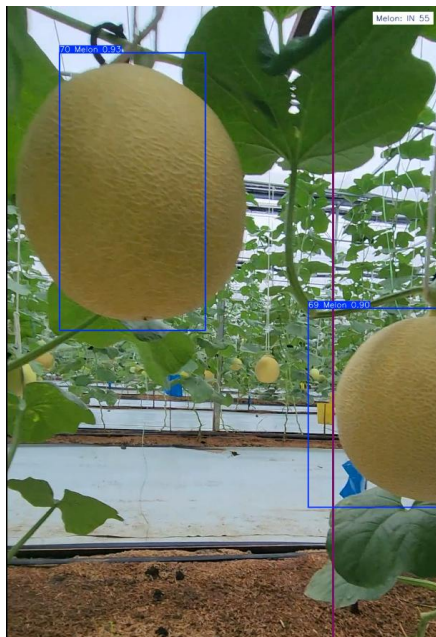


FIG 2. V4R MELON COUNTING

IV. EXPERIMENTAL RESULTS

To fully see what the most impactful parameter between the dataset and the type of process is, multiples cross runs will be done to test all the datasets with all the types of process. Then, a comprehensive analysis was conducted for each dataset and type of process.

A. Datasets Comparisons

The V1D dataset includes annotations for every visible melon in each image, with at least 75 percent of the melon visible, regardless of its position. One strength of the dataset during the tests is its wide coverage of melons in various positions and visibility levels with ease of use thanks to the Roboflow platform. However, it has multiple weaknesses. One is the high number of false positives due to the inclusion of partially obscured melons, another is the inclusion of melons from other parts of the farm that are not currently being treated. The results in Fig 3 shows that this is the least accurate dataset with 5927 melons detected at most, an 8019.17% error

rate. And it only managed to get a 56.16% error rate in the best case with the V4R method. This suggests that while V1D is comprehensive, its inclusivity leads to significant inaccuracies, likely due to the complexity and noise introduced by including partially visible and distant melons.

The V2D dataset focuses exclusively on melons in the front row that are clearly visible and mostly unobstructed by leaves. A notable strength of the V2D dataset is its higher accuracy in detecting clearly visible melons, which reduces false positives. However, it has limited capacity when partially obscured. The V2D dataset showed significant improvements in error rates, particularly when combined with the V4R method, achieving the lowest error rate of 2.74%. This combination also demonstrated a reasonable processing time of 153.09 seconds, making it the most efficient and accurate configuration. At worst, it managed to have a 1953.42% error rate, much lower than the V1D and can be explained by the nature of the process not tracking melons between frames. The focused nature of V2D likely contributes to its superior performance by minimizing the complexity and potential for error in detection and counting.

The V3D dataset classifies melons into two categories: front-melon and rear-melon. A strength of the V3D dataset is its detailed classification by position, which potentially improves detection accuracy. However, in application, it introduces increased complexity in annotation and potential for misclassification. The V3D dataset showed varied results, with the V3R method achieving a moderate error rate of 6.85%, but the V2R method still struggled with a high error rate of 1763.01%. The mixed results suggest that while position-based classification can improve accuracy, it also introduces complexities that may not always lead to better performance.

B. Running Method Comparisons

The V1R running version utilizes the Roboflow hosted model. A strength of the V1R method is its utilization of cloud computing resources, reducing local computational load. However, it has a slow processing time of 52.6 minutes (3157.44 seconds) and requires a stable internet connection. Thus, it is not suitable for real-time usage due to its long processing time. The V1R method, when combined with the V1D dataset, resulted in an extremely high error rate of 8019.17%, making it the least effective combination. Fig 4 show the huge in computation when compared to other solutions. The slow processing time and high error rate indicate that cloud-based processing, while resource-efficient, introduces significant, likely due to communication overhead. It was abandoned as a suitable solution immediately for these reasons.

The V2R version employs the 'predict' method from the Ultralytics library. A strength of the V2R method is its faster processing time compared to V1R, along with local computation. However, as it lacks persistence between frames, the number of duplications when counting the melons make this solution extremely inaccurate as it manages to get 1360 melons in its best run, an 1763,01% error rate. But it is at least faster than the previous method with 202.35s in computation time at the best. The lack of frame persistence likely contributes to the high error rates, as melons can be counted multiple times, leading to significant inaccuracies.

The V3R version uses the 'track' method from the Ultralytics library. A strength of the V3R method is its improved tracking between frames and largely reduced duplication. However, there is a chance for losing track of melons, leading to miscounting. The V3R method demonstrated moderate error rates, with the V2D dataset achieving an error rate of 21.92% and the V3D dataset achieving 6.85%. The processing times were also reasonable and very close to each run, ranging from 184.90 to 188.18 seconds. Showing great stability with any dataset. The improved tracking capabilities of V3R likely contribute to its moderate error rates, but the potential for losing track of melons still shows some inaccuracies and potential improvement.

The V4R version counts melons as they cross a line, ensuring that each melon is counted only once. A strength of the V4R method is its accurate counting with minimal duplication, making it suitable for real-world applications. However, it has a slightly slower processing time compared to V3R, and there is a potential for missing a melon if it is obstructed when passing the line. The V4R method showed the most promising results, particularly when combined with the V2D dataset, achieving the lowest error rate of 2.74% and a processing time of 153.09 seconds. Even with other datasets and when compared to same results with other methods, the V4R method demonstrated reasonable error rates, such as 56.16% with the V1D dataset and 19.18% with the V3D dataset. The line-counting strategy of V4R ensures accurate and consistent counting, making it the most effective method for real-world applications despite its slightly slower processing time.

TABLE 1. RESULTS FROM BENCHMARKS

Melon recognition results			
Version	Speed	Number of melons	Error rate
V1D x V1R	3157.44s	5927	8019,17%
V1D x V2R	242.59s	4709	6350,68%
V1D x V3R	186.57s	319	336,99%
V1D x V4R	170.54s	114	56,16%
V2D x V2R	230.02s	1499	1953,42%
V2D x V3R	188.18s	89	21,92%
V2D x V4R	153.09s	71	2,74%
V3D x V2R	202,35s	1360	1763,01%
V3D x V3R	184.90s	68	6,85%
V3D x V4R	137.47s	59	19,18%

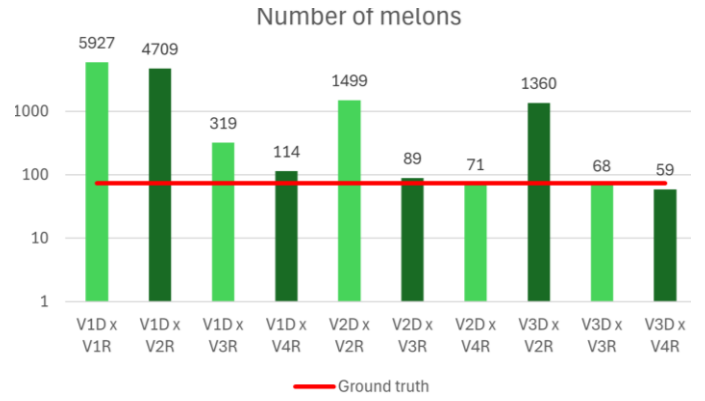


FIG 3. RESULTS IN NUMBER OF MELONS DETECTED

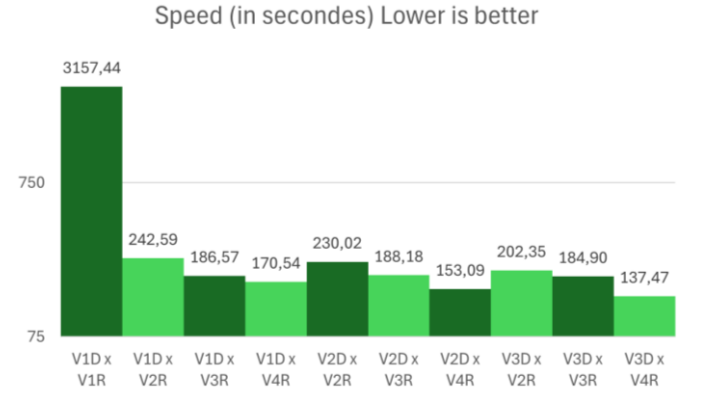


FIG 4. RESULTS IN TERMS OF COMPUTATION SPEED

C. Point of reference

As a point of reference, a benchmark with the basic YOLOv11n model has been made with two of the best running methods, the V3R and the V4R.

When running the model, a few issues were observed. First, YOLOv11 detects the crops as oranges, not melons, and second, the model struggles to keep a melon detected, even when not obstructed.

The results in TABLE 2 show that when used with the V3R running method, it achieved a processing time of 196.26 seconds and detected 112 melons with an error rate of 53.425%. This indicates that while the V3R method is relatively fast, it still struggles with accuracy, likely due to the lack of a specialized dataset tailored to the specific conditions of the melon greenhouse. The error rate suggests that the model overcounts melons, possibly due to tracking issues and misclassifications.

On the other hand, the YOLOv11n model combined with the V4R running method showed a slightly faster processing time of 186.65 seconds but detected significantly fewer melons, with a count of 37 for an error rate of 49.315%. This undercounting could be attributed to the line-counting strategy of V4R and to the model struggling to detect "oranges" even without obstruction.

Overall, the benchmark results with the basic YOLOv11n model highlight the importance of using specialized datasets and advanced running methods to achieve accurate, constant, and more efficient melon detection and counting. The custom datasets developed above significantly outperform the basic YOLOv11n model and serve to add correct labelling to the subjects.

TABLE 2. RESULTS WITH YOLOv11N

Version	Speed	Number of melons	Error rate
YOLOv11n x V3R	196,26	112	53,425
YOLOv11n x V4R	186,65	37	49,315

V. CONCLUSION

In conclusion, this study shows it is possible to have an automated crop detection and counting system in a non-lab environment while having a minimal error rate, especially when combining a strict detection dataset with a line counting system to ensure no duplication and accurate counting. It also shows the importance to have custom datasets to ensure smooth operation and correct labelling.

The results obtained point to several areas for improvement. First, the tracking could be upgraded to better handle temporary occlusions. Second, the time taken to compute is almost twice as long as the original video, such a long time makes the actual system unable to be used in real time. In third, with only 392 images for training, the datasets are considered small. A small dataset can limit the results and tend gave worse results, higher computation time due to having more difficulties when rare cases or partially obscured cases occurred. A larger dataset could help for computation speed and accuracy with a bonus point to demonstrate the practical usage if the larger dataset make use of different environments.

Furthermore, the practical deployment of such a system raises the question of infrastructure design: should the processing be performed locally on the device capturing the video, on a dedicated server within the farm, or through a cloud-based solution? Each of these options presents trade-offs in terms of latency, operational costs, connectivity requirements, and scalability.

REFERENCES

- [1] X. Zan *et al.*, "Tobacco Maturity Recognition Based on an Improved YOLOv8 Algorithm," *Proceedings - 2023 5th International Symposium on Smart and Healthy Cities, ISHC 2023*, pp. 58–62, 2023, doi: 10.1109/ISHC61216.2023.00019.
- [2] L. Zhou, H. Gao, D. Gao, and Q. Zhao, "Recognition of Tibetan Medicinal Plants Based on Local Feature Localization," *2024 IEEE 5th International Conference on Pattern Recognition and Machine Learning, PRML 2024*, pp. 403–407, 2024, doi: 10.1109/PRML62565.2024.10779871.
- [3] M. Faisal, F. Albogamy, H. Elgibreen, M. Algabri, and F. A. Alqershi, "Deep Learning and Computer Vision for Estimating Date Fruits Type, Maturity Level, and Weight," *IEEE Access*, vol. 8, pp. 206770–206782, 2020, doi: 10.1109/ACCESS.2020.3037948.
- [4] X. Liu, W. Min, S. Mei, L. Wang, and S. Jiang, "Plant Disease Recognition: A Large-Scale Benchmark Dataset and a Visual Region and Loss Reweighting Approach," *IEEE Transactions on Image Processing*, vol. 30, pp. 2003–2015, 2021, doi: 10.1109/TIP.2021.3049334.
- [5] S. Barburiceanu, S. Meza, B. Orza, R. Malutan, and R. Terebes, "Convolutional Neural Networks for Texture Feature Extraction. Applications to Leaf Disease Classification in Precision Agriculture," *IEEE Access*, vol. 9, pp. 160085–160103, 2021, doi: 10.1109/ACCESS.2021.3131002.

- [6] S. C. Hsia, S. H. Wang, J. Y. Yeh, and C. Y. Chang, "A Smart Leaf Blow Robot Based on Deep Learning Model," *IEEE Access*, vol. 11, pp. 111956–111962, 2023, doi: 10.1109/ACCESS.2023.3307136.
- [7] J. U. M. Akbar, S. F. Kamarulzaman, A. J. M. Muzahid, M. A. Rahman, and M. Uddin, "A Comprehensive Review on Deep Learning Assisted Computer Vision Techniques for Smart Greenhouse Agriculture," *IEEE Access*, vol. 12, pp. 4485–4522, 2024, doi: 10.1109/ACCESS.2024.3349418.
- [8] "Roboflow," <https://roboflow.com/>.
- [9] "Ultralytics," <https://github.com/ultralytics/ultralytics>.